

AutoFlow AI: An Agent-Based Workflow Automation System Using Large Language Models

Prof. Sumedha Maindalkar

Department of Artificial Intelligence and Machine Learning
SIES Graduate School of Technology
Nerul, Mumbai, India
sumedham@sies.edu.in

Abuzaid Khan

Department of Artificial Intelligence and Machine Learning
SIES Graduate School of Technology
Nerul, Mumbai, India
abuzaidk1234@gmail.com

Karan Chand

Department of Artificial Intelligence and Machine Learning
SIES Graduate School of Technology
Nerul, Mumbai, India
karanchand0103@gmail.com

Shrey Mhatre

Department of Artificial Intelligence and Machine Learning
SIES Graduate School of Technology
Nerul, Mumbai, India
shreymhatre9@gmail.com

Abstract—AutoFlow AI is an intelligent workflow automation system that converts natural language instructions into executable automated tasks using Large Language Models (LLMs). Traditional automation systems require manual configuration and predefined triggers, which limits flexibility and usability for non-technical users. The proposed system introduces an AI-driven architecture that integrates natural language understanding, task scheduling, decision-making, and automated execution within a unified framework.

The system is built on a desktop-plus-service architecture consisting of an Electron desktop client, a FastAPI backend, and a LangGraph-based execution runtime. Tool access is divided between native in-process tools, which handle task scheduling and knowledge retrieval, and MCP (Model Context Protocol) tools, which handle external actions such as email delivery. The runtime supports two execution modes: local inference through Qwen3:4b via Ollama, and cloud-backed inference through Gemini 2.5 Flash. Organizational documents are indexed into a Chroma vector store for retrieval-augmented responses.

Evaluation across five scenario categories—email automation, knowledge-based file sharing, scheduled task execution, deterministic tool invocation, and real-time web search—demonstrates that the system reliably connects natural language understanding with practical automation functions. The architecture supports dynamic tool registration through the Model Context Protocol, allowing new external services to be connected at runtime without modifying application code.

Index Terms—Workflow Automation, Large Language Models, Agent-Based Systems, Natural Language Processing, Retrieval-Augmented Generation, Task Scheduling, Model Context Protocol

I. INTRODUCTION

Workflow automation systems are widely used to reduce manual effort in tasks such as email handling, report generation, and data processing. Platforms such as Zapier and n8n allow users to connect applications through predefined trigger-action rules. However, configuring these rules requires technical knowledge and cannot handle instructions that arrive in unstructured natural language.

Large Language Models have shown strong capabilities in language understanding, reasoning, and tool use [1]. These properties make them suitable as reasoning cores for automation systems that accept natural language input and translate it into structured actions. Prior work such as ReAct [1] and Toolformer [2] demonstrated that LLMs can interleave reasoning steps with tool invocations, enabling multi-step task completion without human intervention at each step.

Building on these foundations, this paper describes AutoFlow AI, a working prototype of an agent-based workflow automation system. The system accepts natural language instructions through an Electron desktop client, routes them through a FastAPI backend and a LangGraph execution runtime, and delivers results through a set of native and MCP-registered tools. The described system is a functional implementation with verifiable outputs including sent emails, delivered attachments, executed scheduled tasks, and real-time web search responses.

The main contributions of this work are as follows:

- A modular desktop-plus-service architecture is implemented that separates the user interface, backend API, execution runtime, tool layer, and data layer into distinct components.
- A dual execution mode is supported, allowing users to switch between local inference using Qwen3:4b through Ollama and cloud inference using Gemini 2.5 Flash through the Google AI API, with automatic fallback when cloud credentials are unavailable.
- A knowledge retrieval layer is integrated using Chroma and a watchdog-based file monitor, enabling retrieval-augmented responses grounded in organizational documents.
- Dynamic tool extensibility is achieved through the Model Context Protocol [5], allowing STDIO and SSE servers to be registered at runtime without restarting the application.
- A background task scheduler is implemented using AP-

Scheduler [14], enabling deferred and recurring workflow execution without requiring the user to remain active in the chat interface.

II. RELATED WORK

A. Reasoning and Acting in LLM Agents

Yao *et al.* introduced ReAct [1], a prompting strategy that interleaves reasoning traces with tool-use actions in a single generation loop. The approach allows an LLM to consult external sources, revise its plan based on retrieved information, and proceed to the next action. On the ALFWorld and WebShop benchmarks, ReAct outperformed imitation and reinforcement learning baselines. AutoFlow AI applies a similar reasoning-action loop through LangGraph, which maintains thread-scoped state across tool invocations within each user session.

B. Tool Use in Language Models

Schick *et al.* proposed Toolformer [2], a self-supervised training approach in which a language model learns to insert API calls into generated text by selecting calls that reduce prediction loss on future tokens. The model was trained with a calculator, a search engine, a question-answering system, a translation system, and a calendar. AutoFlow AI addresses a related problem at the deployment level: rather than training the model to call APIs, the system registers tools through the Model Context Protocol and relies on the model’s in-context reasoning to select the appropriate tool at inference time.

C. Retrieval-Augmented Generation

Lewis *et al.* introduced RAG [3], combining a pre-trained sequence-to-sequence model with a dense passage retriever over Wikipedia. Retrieval grounds generation in external documents and reduces the model’s dependence on parametric knowledge alone. AutoFlow AI applies the same principle at the organizational level: shared documents stored in a watched directory are chunked, embedded, and indexed into Chroma. When a user query requires document content, the `search_knowledge_base` tool retrieves the most relevant chunks and returns them with source metadata.

D. Automated Workflow Generation

Li *et al.* proposed AutoFlow [4], a framework that represents agent workflows as natural language programs and uses a workflow optimization procedure to iteratively improve workflow quality. The present work addresses a complementary problem: rather than optimizing the workflow structure automatically, AutoFlow AI provides a fixed orchestration loop and allows the user to specify intent in natural language, relying on the model’s reasoning to select tools and order actions.

E. Tool Integration Protocols

Anthropic released the Model Context Protocol in November 2024 [5] as an open, vendor-neutral protocol for connecting LLM applications to external tools and data sources. MCP defines a client-server interaction model in which the AI

application acts as a client and tool providers act as servers, communicating over STDIO or SSE transports. AutoFlow AI uses MCP as its primary mechanism for integrating external services; by default it loads email and math servers, and additional servers can be registered without modifying the backend source code.

III. SYSTEM ARCHITECTURE

AutoFlow AI follows a desktop-plus-service architecture. The Electron desktop client [6] provides the login screen, chat interface, knowledge panel, scheduled task view, MCP server manager, and settings panel. The backend is implemented with FastAPI [7] and exposes endpoints for authentication, streaming chat, file management, task management, user preferences, and MCP configuration.

The execution runtime is built on LangGraph [8], an open-source Python library for constructing stateful, multi-actor LLM workflows. LangGraph represents workflows as directed graphs where each node is a callable that reads from and writes to a shared state object. The runtime uses LangGraph’s in-memory checkpointing to maintain thread-scoped conversation history across multiple turns within the same session.

Tool access is divided into two groups. Native in-process tools are defined directly in the backend and include `schedule_task`, which writes deferred jobs to the SQLite task table, and `search_knowledge_base`, which retrieves document chunks from the Chroma index. MCP tools are loaded from configured servers at request time; by default the system loads an email server and a math server over STDIO. Additional STDIO or SSE servers can be registered at runtime through the MCP management API [5].

The Electron main process also starts a local filesystem MCP node using Express and SSE after login. This node exposes `list_directory`, `read_file`, and `write_file` operations scoped to the local `AutoFlowData` directory. When external access is required, ngrok [13] publishes an SSE endpoint for that node and displays the resulting MCP URL in the interface.

Persistent state is stored in SQLite. The database holds user accounts, password hashes, SMTP credentials, Gemini API keys, preferred model settings, and the scheduled task queue. Shared documents are stored in the backend `org_filesystem` directory and indexed into a Chroma [9] collection for retrieval. A watchdog observer monitors the directory so that newly added, modified, or deleted files are reflected in the index automatically.

The runtime supports two execution modes. In local mode, requests are processed by Qwen3:4b [11] running through Ollama [10]. In cloud mode, requests are processed by Gemini 2.5 Flash [12] through the Google AI API when a valid key is available; otherwise the system falls back to the local model automatically.

IV. METHODOLOGY

The runtime pipeline of AutoFlow AI is organized into four stages: startup preparation, user session setup, interactive request execution, and deferred task execution.

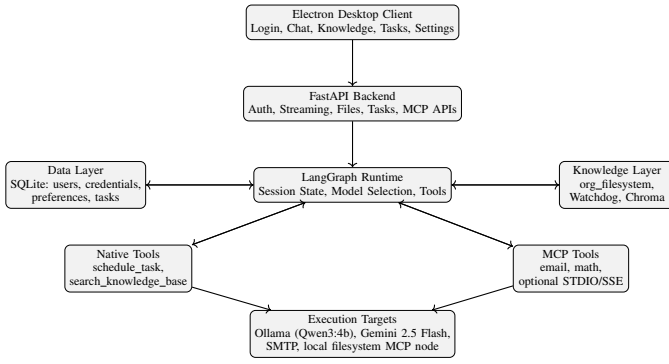


Fig. 1. Implemented architecture of AutoFlow AI. Bidirectional arrows indicate request and response flows between components.

A. Startup Preparation

When the backend starts, it initializes the SQLite schema and creates required directories. It then scans the shared `org_filesystem` directory and indexes supported file types—PDF, DOCX, TXT, MD, and common source-code formats—into Chroma using a recursive text splitter. Each chunk is stored with filename metadata to support source attribution in retrieval results [9]. A watchdog observer monitors the same directory so changes are reflected in the index without restarting the service.

The startup phase also initializes an AsyncIO-based APScheduler instance [14]. The scheduler polls the task table every ten seconds, checking for records whose `execution_time` is at or before the current UTC timestamp and whose status is pending.

B. User Session Setup

On login, the Electron client authenticates the user through the FastAPI service and obtains a new thread identifier. It retrieves the user’s saved SMTP settings, Gemini API key, and preferred execution mode from the backend database. These settings are stored per user in SQLite and applied at request time, not hardcoded into the application.

After login, the client starts a local filesystem MCP node implemented with Express and SSE, restricted to the local `AutoFlowData` directory. If external access is needed, ngrok [13] exposes the SSE endpoint and the resulting URL is displayed in the MCP server panel.

C. Interactive Request Execution

When the user submits a message, the client sends the message text, thread identifier, username, and preferred model to the backend `/agent/stream` endpoint. The backend loads the active MCP server configuration for that user, then initializes a LangGraph run for the specified thread [8]. Thread-level message history is preserved through LangGraph’s in-memory checkpointing, enabling multi-turn conversations within the same session.

Model selection is resolved at request time. If the user has selected cloud mode and a valid Gemini API key is present, the backend uses Gemini 2.5 Flash [12]; otherwise it uses

Qwen3:4b through Ollama [10], [11]. This fallback ensures the system remains operational without cloud connectivity.

The active toolset combines native tools with the currently loaded MCP tools. If the request involves shared documents, the runtime invokes `search_knowledge_base`, which queries Chroma and returns the most relevant chunks with source metadata. For external actions, the runtime invokes the appropriate MCP tool. The backend streams the run to the client as newline-delimited JSON events containing the active model name, intermediate tool-use markers, and assistant message content, which is why the interface can display tool badges such as `send_email` or `schedule_task` during response generation.

The backend also includes a normalization step for models that emit tool requests as plain text rather than structured tool calls. When this pattern is detected, the JSON block is converted into a formal tool call and the runtime is reinvoked with the corrected input. This improves reliability without changing the user-facing flow.

D. Scheduling and Background Execution

Deferred execution is handled by the `schedule_task` native tool. When the user requests a future action, the tool parses the target time using `dateparser` [15] and converts it to UTC for storage. The task record includes the original prompt, a rephrased imperative prompt, the execution timestamp, optional metadata, and an optional recurrence rule. When APScheduler finds a due task, it marks the record as processing and invokes the same LangGraph workflow used for interactive requests, but with a system instruction that directs the model to execute the action immediately rather than ask clarifying questions. Tool outputs and assistant messages are written to the task log and displayed in the Scheduled Tasks panel. If the task has a recurrence rule, the next instance is inserted into the task table after successful execution.

V. ALGORITHM

Algorithm 1 Interactive Request Handling in AutoFlow AI

```

1: Receive (message, thread_id, username, preferred_model)
   from the Electron client
2: Validate the user; load preferences, SMTP credentials,
   Gemini key, and MCP configuration from SQLite
3: if preferred_model is cloud and a valid Gemini API key
   exists then
4:   Select Gemini 2.5 Flash via the Google AI API
5: else
6:   Select Qwen3:4b through Ollama
7: end if
8: Build active toolset: native tools  $\cup$  configured MCP tools
9: Initialize LangGraph execution state for the given
   thread_id
10: if the request specifies future execution then
11:   Parse target time using dateparser; convert to UTC
12:   Store task record in scheduled_tasks with status
   pending
13:   Return scheduling confirmation to the client
14: else
15:   if the request requires shared organizational knowledge
   then
16:     Invoke search_knowledge_base; retrieve top-k
     chunks from Chroma
17:   end if
18:   Invoke the LangGraph runtime with the selected model
   and active toolset
19:   if the model emits a raw JSON-like tool intent instead
   of a structured call then
20:     Parse the JSON block into a structured tool call
21:     Reinvoke the runtime with the corrected tool call
22:   end if
23:   Stream model_info, tool_use, and message
   events to the client
24: end if

```

Algorithm 2 Scheduled Task Polling and Background Execution

```

1: Every 10 seconds, query scheduled_tasks for
   records where status = pending and execution_time  $\leq$ 
   current UTC time
2: for each due task do
3:   Mark the task as processing
4:   Load the task owner's preferred model, Gemini key,
   SMTP settings, and MCP configuration
5:   Construct background prompt from stored rephrased
   imperative description
6:   Invoke the LangGraph workflow in autonomous mode
7:   Append tool outputs and assistant messages to the task
   log
8:   if execution succeeds then
9:     Mark the task as completed
10:    if the task has a recurrence rule then
11:      Compute next run time; insert new pending task
12:    end if
13:  else
14:    Mark the task as failed; store error message in the
    task log
15:  end if
16: end for

```

VI. RESULTS AND DISCUSSION

The AutoFlow AI prototype was evaluated through five representative workflow scenarios. Each scenario exercises a distinct capability of the system and is validated by a verifiable output.

A. Email Automation

When the user submits a natural language email request, the LangGraph runtime selects the `send_email` MCP tool, constructs the required arguments from the message, and returns a confirmation after delivery. Fig. 2 shows the chat interaction and the `send_email` tool badge displayed during execution. Fig. 3 shows the received email, confirming end-to-end delivery from natural language input to inbox.

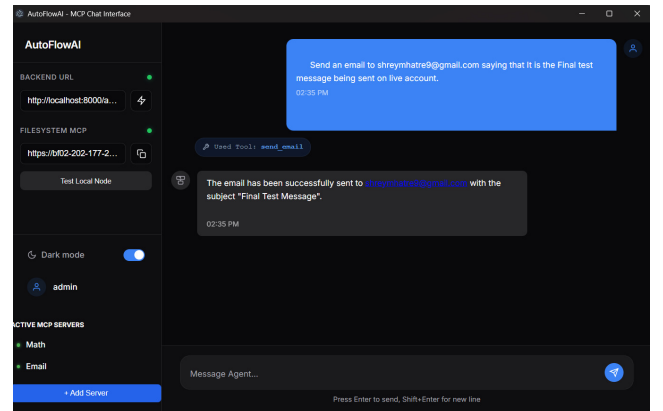


Fig. 2. Natural language email request executed through the `send_email` MCP tool.



Fig. 3. Received email confirming successful end-to-end execution of the email workflow.

B. Knowledge-Based File Sharing via Email

When the user requests that a specific document be sent to a recipient, the runtime first invokes `search_knowledge_base` to locate the file in the Chroma index, then invokes `send_email` to deliver it as an attachment. This scenario requires the model to coordinate two tools in sequence without explicit user guidance. Fig. 4 shows the two-tool interaction in the chat interface. Fig. 5 shows the received email with the attached document, validating that retrieval and delivery both succeeded.

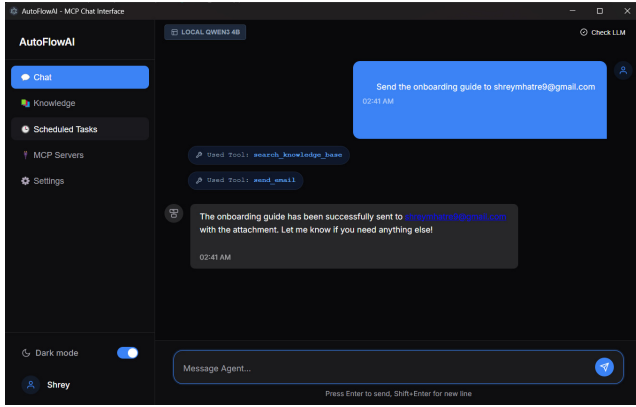


Fig. 4. Knowledge-based file sharing using `search_knowledge_base` followed by `send_email`.

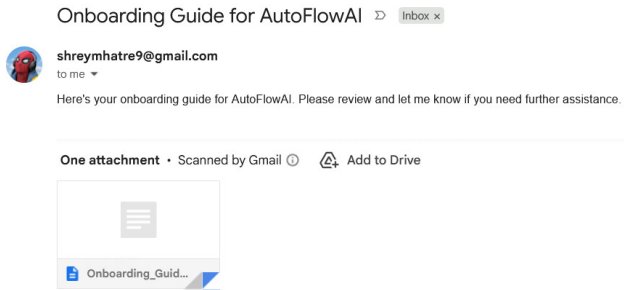


Fig. 5. Received email confirming successful delivery of the retrieved document attachment.

C. Scheduled Task Automation

When the user requests an action to occur at a future time, the `schedule_task` tool parses the target time, converts it to UTC, and stores a task record with status pending. APScheduler then polls the task table every ten seconds and executes the task when its time arrives, without further user interaction. Fig. 6 shows the natural language scheduling request and the scheduling confirmation. Fig. 7 shows the Scheduled Tasks panel after execution, displaying the task log entry and the completed status.

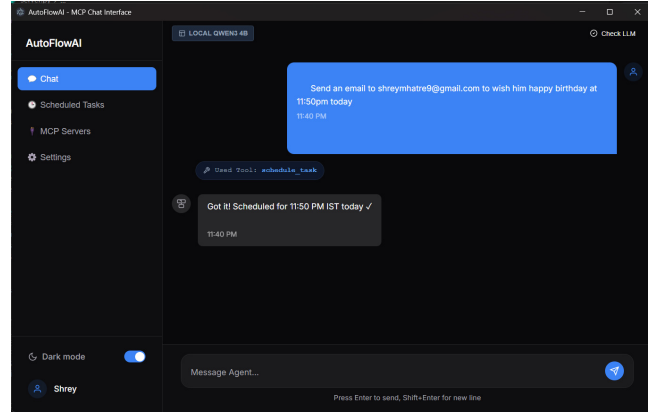


Fig. 6. Natural language request converted into a deferred task through `schedule_task`.

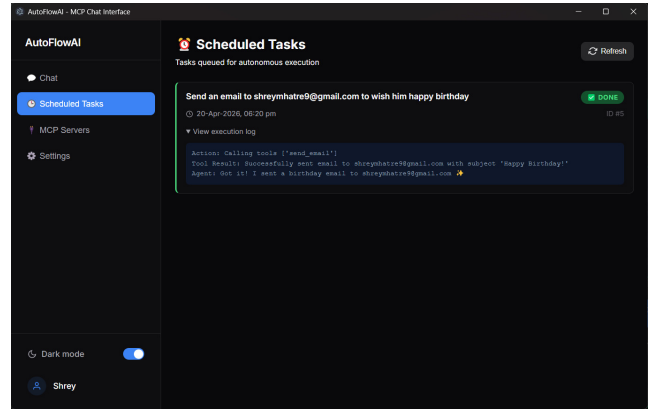


Fig. 7. Scheduled Tasks panel showing completed status and task log after autonomous execution.

D. Tool-Routed Responses

The runtime routes different query types to specialized tools rather than relying on model generation alone. For arithmetic queries, the system invokes the `add MCP math` tool to produce a deterministic result, as shown in Fig. 8. For real-time informational queries, the system invokes the `web_search` tool to retrieve current information, as shown in Fig. 9.

E. System Configuration and Monitoring

The settings interface in Fig. 10 allows users to select between local and cloud execution modes and to store their

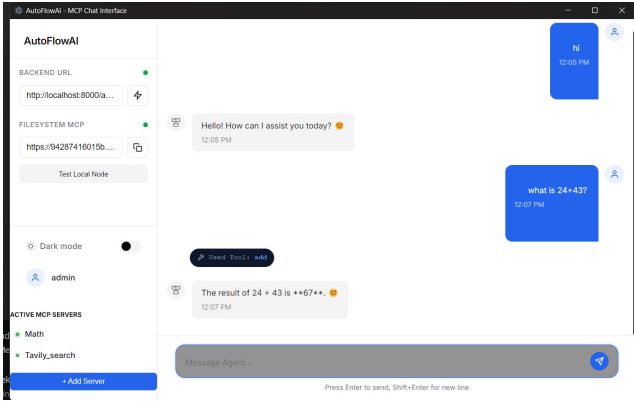


Fig. 8. Arithmetic query routed to the add MCP math tool for deterministic computation.

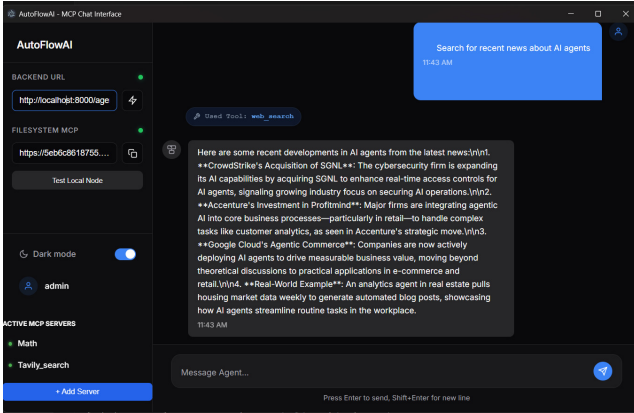


Fig. 9. Informational query answered using the web_search tool.

Gemini API key and SMTP credentials. These values are saved to the per-user SQLite record and applied at the next request. The dashboard in Fig. 11 shows the currently active model, running thread count, and MCP server connectivity. The MCP server management panel in Fig. 12 allows users to register additional STDIO or SSE servers at runtime without restarting the application.

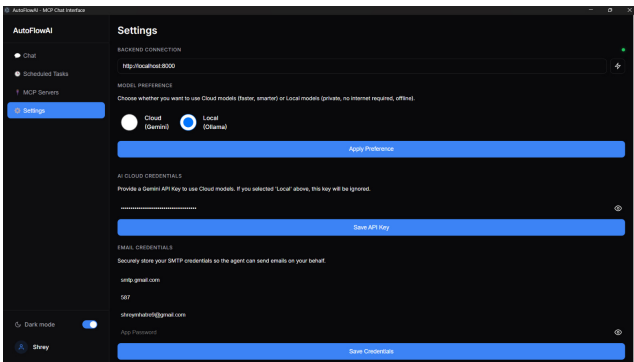


Fig. 10. Settings interface for model selection and credential management.

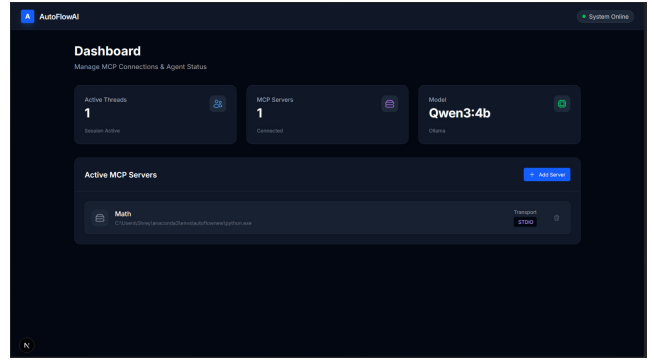


Fig. 11. Dashboard showing active model, thread count, and MCP server connectivity.

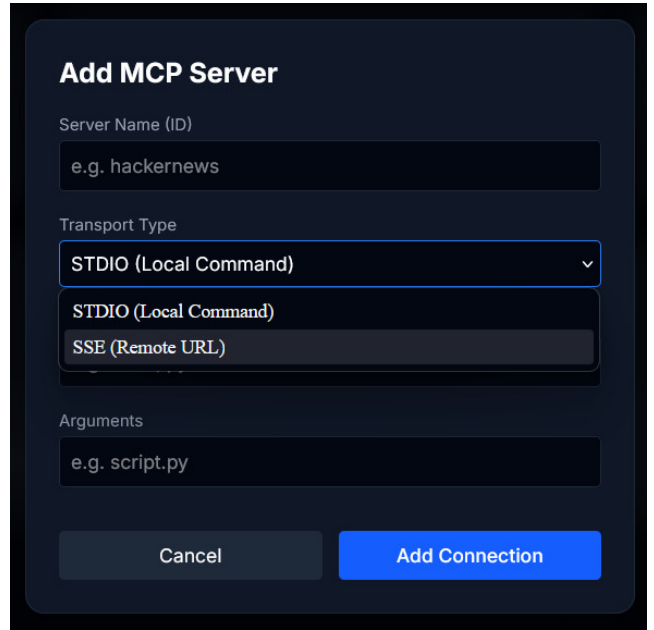


Fig. 12. MCP server management panel for registering additional external tool connections at runtime.

F. Discussion

The five scenarios collectively demonstrate that AutoFlow AI can connect natural language input to concrete, verifiable outputs: sent emails, delivered attachments, autonomously executed scheduled tasks, deterministic arithmetic results, and real-time web search responses. The dual execution mode—local Qwen3:4b through Ollama and cloud Gemini 2.5 Flash—allows the system to remain operational in environments without internet connectivity.

The current evaluation is scenario-based. Future work will address quantitative measurement across a broader task set, latency profiling for both execution modes, and an assessment of scheduling accuracy under concurrent task loads.

VII. CONCLUSION

This paper presented AutoFlow AI, an agent-based workflow automation system that converts natural language instruc-

tions into executable workflows using Large Language Models. The system is implemented as a working prototype with an Electron desktop client, a FastAPI backend, a LangGraph execution runtime, native scheduling and retrieval tools, and a dynamically extensible MCP tool layer.

Prototype evaluation demonstrated reliable performance across email automation, knowledge-based file sharing, scheduled task execution, deterministic tool use, and real-time information retrieval. The architecture supports both private local inference through Ollama and cloud inference through the Google Gemini API, with automatic fallback between modes.

Future work will focus on quantitative benchmarking across diverse task categories, support for multi-agent coordination within the LangGraph runtime, and integration with enterprise identity and access management systems.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [2] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2302.04761>
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [4] Z. Li, S. Xu, K. Mei, W. Hua, B. Rama, O. Raheja, H. Wang, H. Zhu, and Y. Zhang, “AutoFlow: Automated workflow generation for large language model agents,” arXiv preprint arXiv:2407.12821, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2407.12821>
- [5] Anthropic, “Model Context Protocol: An open protocol for connecting AI models to external tools and data sources,” released Nov. 2024. [Online]. Available: <https://modelcontextprotocol.io/specification>. Accessed: Apr. 21, 2026.
- [6] Electron, “Introduction,” Electron Documentation. [Online]. Available: <https://www.electronjs.org/docs/latest>. Accessed: Apr. 21, 2026.
- [7] FastAPI, “FastAPI,” FastAPI Documentation. [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: Apr. 21, 2026.
- [8] LangChain AI, “LangGraph: A library for building stateful, multi-actor applications with LLMs,” 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph>. Accessed: Apr. 21, 2026.
- [9] Chroma, “Introduction,” Chroma Docs. [Online]. Available: <https://docs.trychroma.com/>. Accessed: Apr. 21, 2026.
- [10] Ollama, “Ollama documentation.” [Online]. Available: <https://docs.ollama.com/>. Accessed: Apr. 21, 2026.
- [11] Ollama, “Qwen3,” Ollama Model Library. [Online]. Available: <https://ollama.com/library/qwen3>. Accessed: Apr. 21, 2026.
- [12] Google DeepMind, “Gemini 2.5 Flash,” Google AI for Developers. [Online]. Available: <https://ai.google.dev/gemini-api/docs/api-overview>. Accessed: Apr. 21, 2026.
- [13] ngrok, “What is ngrok?,” ngrok Documentation. [Online]. Available: <https://ngrok.com/docs/>. Accessed: Apr. 21, 2026.
- [14] APScheduler Project, “User guide,” AP-Scheduler Documentation. [Online]. Available: <https://apscheduler.readthedocs.io/en/master/userguide.html>. Accessed: Apr. 21, 2026.
- [15] dateparser, “dateparser – python parser for human readable dates.” [Online]. Available: <https://dateparser.readthedocs.io/>. Accessed: Apr. 21, 2026.